AWS INTERVIEW QUESTIONS & ANSWERS

Interview-ready answers for a candidate with approximately 3 years of AWS experience

Topics: EC2 • VPC • S3 • IAM • Load Balancing • RDS • CloudWatch • SNS • CloudTrail

Prepared from the questions in your attached document. Answers use a practical, production-oriented interview style.

Contents

- EC2
- VPC
- S3
- IAM
- Load Balance
- RDS
- Cloud Watch
- SNS
- Cloud trail

How to use this guide: explain the approach first, then mention the AWS services and checks you would use. In real interviews, be clear about what you have personally implemented versus what you understand conceptually.

EC2

Q1. What is an EC2 instance?

Answer: Amazon EC2 (Elastic Compute Cloud) provides resizable virtual servers in AWS. An instance is a running virtual machine whose CPU, memory, storage, networking, operating system, and capacity are selected for the workload. I choose an AMI, instance type, VPC/subnet, security group, key pair or approved access method, storage, and IAM role when launching it.

Q2. How do you deploy an EC2 instance?

Answer: I first confirm the application requirements and environment. In the correct AWS Region and VPC, I select a trusted AMI and suitable instance type, configure subnet and security group with least-privilege ports, attach an IAM instance profile, configure encrypted EBS storage, and use approved key-based or Systems Manager access. I launch the instance, install/configure the application using a repeatable script or image, deploy the build, configure health checks and logs, test connectivity, and document/monitor the deployment. For production, I prefer infrastructure as code and a launch template/Auto Scaling group rather than manual-only setup.

Q3. An EC2 instance is not reachable. What will you check?

Answer: I troubleshoot layer by layer: (1) confirm instance state, status checks, correct public/private IP and DNS; (2) check security-group inbound/outbound rules; (3) check subnet route table, internet gateway or NAT path as appropriate, and network ACL rules in both directions; (4) verify the client source IP and correct SSH/RDP port; (5) confirm the OS firewall, service status, CPU/memory/disk, and application listening port; (6) check key pair/username or Systems Manager permissions. I use EC2 system/instance logs, console output, Reachability Analyzer, VPC Flow Logs and CloudWatch metrics where relevant. A private instance normally requires VPN/Direct Connect, a bastion, or Systems Manager—not a public IP.

Q4. How do you perform zero-to-minimal downtime deployment?

Answer: I use a deployment strategy that keeps healthy capacity serving traffic. With a load balancer and Auto Scaling group, a common option is blue/green: deploy and test the new version in a separate target group or environment, run health checks and smoke tests, then shift traffic gradually or switch the listener. Another option is rolling deployment, replacing instances in batches while maintaining minimum healthy capacity. I keep database changes backward-compatible, use automated rollback/alarms, monitor errors and latency, and retain the previous version until the new release is stable. The exact approach depends on the application and its stateful dependencies.

Q5. Security group vs NACL

Answer: A security group is a stateful virtual firewall attached to network interfaces/instances: allow rules only, and return traffic is automatically permitted for established connections. A network ACL is a stateless subnet-level filter: it supports allow and deny rules, evaluates rules in number order, and requires inbound and outbound rules for both sides of a connection. I use security groups as the primary workload-level control and NACLs for broader subnet guardrails where needed.

Q6. How do you troubleshoot an application that is running?

Answer: I check whether the instance and service are healthy, then reproduce the issue and inspect application logs, system logs and recent deployment/configuration changes. I review CPU, memory, disk space/inodes, network connections, process status, ports, DNS, dependencies and database connectivity. I use CloudWatch metrics/alarms and centralized logs, plus health endpoints and load-balancer target health. I compare a healthy instance with the failing one, mitigate customer impact first, and then fix the root cause and document it. I avoid restarting services blindly before capturing useful evidence.

VPC

Q7. Explain your production VPC architecture.

Answer: A typical production design uses a VPC across at least two Availability Zones. Public subnets host internet-facing load balancers and, if required, NAT gateways. Application EC2 instances or containers run in private subnets, and databases run in isolated/private database subnets. The internet gateway is attached to the VPC; public route tables route internet-bound traffic to it, while private-subnet outbound internet traffic uses a NAT gateway in a public subnet. Security groups restrict traffic between tiers, IAM controls AWS API access, and VPC Flow Logs, CloudTrail and CloudWatch support monitoring and audit. I also plan CIDR ranges, DNS, endpoints, backups, high availability and connectivity to corporate networks.

Q8. Why do we use public and private subnets?

Answer: A public subnet has a route to an internet gateway; a resource also needs a public IPv4 address or other suitable public endpoint configuration to communicate directly with the internet. It is commonly used for internet-facing load balancers and NAT gateways. A private subnet does not have a direct route to the internet gateway for its workloads. It is used for application servers and databases to reduce exposure. Private resources can still reach required services through NAT gateways, VPC endpoints, or private network connectivity.

Q9. How can a private EC2 instance connect to the internet?

Answer: For outbound IPv4 internet access, I place a NAT gateway in a public subnet, ensure that subnet has a route to the internet gateway, and add a default route from the private subnet's route table to the NAT gateway. The instance does not need a public IP. I verify security-group egress, NACL return-path rules, DNS and NAT gateway health. For AWS services such as S3, I may use a VPC endpoint to avoid routing that traffic through NAT. NAT gateways do not allow unsolicited inbound internet connections to the private instance.

Q10. What is the difference between a NAT gateway and an internet gateway?

Answer: An internet gateway attaches to a VPC and enables communication between the VPC and the internet for resources with suitable public addressing and routing. A NAT gateway enables instances in private subnets to initiate outbound IPv4 connections to external destinations without exposing those instances to unsolicited inbound connections. A NAT gateway is placed in a public subnet and routes through an internet gateway for internet access. They serve different purposes and are not interchangeable.

Q11. How can two VPCs communicate?

Answer: Common options are VPC Peering, AWS Transit Gateway, or private connectivity through VPN/Direct Connect depending on scale and network design. For VPC Peering, I create the peering connection, accept it if required, add routes in both VPC route tables for the peer CIDR, and allow the required traffic in security groups/NACLs. CIDR ranges must not overlap. Peering is not transitive, so a third VPC cannot automatically route through a peered VPC. Transit Gateway is often better for many VPCs and centralized routing.

Q12. A route table is not working as expected. How do you troubleshoot?

Answer: I identify the source and destination CIDRs and inspect the route table actually associated with the subnet. I check the most-specific matching route, target state (IGW, NAT gateway, peering, Transit Gateway, or endpoint), and whether the route is active or a blackhole. I verify routes on both sides of a connection, subnet associations, overlapping CIDRs, security groups, NACLs, and return routing. I use Reachability Analyzer and VPC Flow Logs to narrow down the path, then test again after making the smallest controlled change.

S3

Q13. An S3 bucket accidentally became public. How do you secure it?

Answer: I immediately enable all four S3 Block Public Access settings at the account and bucket levels where appropriate, review and remove public bucket policies and ACL grants, and check access-point policies. I verify Object Ownership settings and inspect CloudTrail/data events or access logs if enabled to assess possible exposure. I identify whether any public access was intentional, notify the security team, rotate exposed credentials or data where relevant, and review IAM permissions. I validate access from an unauthorised context and add preventive controls/alerts to detect future policy changes.

Q14. What are S3 Versioning and Lifecycle policies, and why use them?

Answer: Versioning preserves multiple versions of an object, helping recover from accidental overwrites or deletes; a delete marker can hide the current version while older versions remain. It should be paired with lifecycle rules to manage storage cost. Lifecycle policies can transition eligible objects to lower-cost storage classes and expire current or noncurrent versions after defined periods. I define retention based on business, recovery and compliance requirements, and remember that versioning can increase storage cost and lifecycle actions may have minimum-duration or retrieval-cost implications.

Q15. How do you encrypt data in an S3 bucket?

Answer: I enable default server-side encryption. SSE-S3 uses S3-managed keys; SSE-KMS uses AWS KMS keys and gives more control over key policies, permissions and auditability. I choose based on security and compliance requirements, restrict who can access objects and use keys, and enforce encryption requirements through bucket policies where needed. For especially sensitive data, client-side encryption may also be considered. Encryption at rest does not replace TLS in transit or least-privilege access controls.

Q16. Large S3 files upload/download slowly. What would you do?

Answer: I first measure whether the bottleneck is client bandwidth, network path, request pattern, object size, region distance or application processing. For large uploads I use multipart upload, which allows parallel parts and retrying failed parts; for downloads I consider byte-range requests and parallel transfer. I use the AWS CLI or SDK transfer configuration, keep clients close to the bucket's Region, and consider S3 Transfer Acceleration for geographically distant clients after testing cost and benefit. I also check throttling/errors, use CloudWatch/S3 request metrics where enabled, and use CloudFront for frequently accessed downloads where appropriate.

IAM

Q17. What is the difference between IAM users, roles and policies?

Answer: An IAM user is an identity with long-term credentials, although workforce access is generally better handled through federation/Identity Center. An IAM role is an identity with permissions that trusted principals can assume to obtain temporary credentials. A policy is a JSON document defining allowed or explicitly denied actions on resources, optionally under conditions; policies are attached to users, groups or roles, or applied as resource-based policies. I follow least privilege, prefer temporary credentials, and avoid hard-coded access keys.

Q18. How do you give an EC2 application permissions to access AWS services?

Answer: I create a narrowly scoped IAM role with a trust policy that allows the EC2 service to assume it, attach only the required permissions policy, and associate the role through an instance profile with the EC2 instance or launch template. The application uses the AWS SDK/CLI credential provider chain to obtain temporary credentials automatically. I do not store access keys in code or user data. I test the exact required actions and use CloudTrail to audit API calls.

Q19. How do you use AssumeRole in a real-time process?

Answer: I configure a role with a trust policy that identifies the trusted account, service or identity allowed to assume it, and attach the permissions needed by the role. The caller invokes STS AssumeRole and receives temporary access key, secret key and session token, which expire after the configured duration. This is useful for cross-account access, CI/CD deployment roles, and delegated administration. I restrict trust and permissions, use external IDs for appropriate third-party scenarios, and log activity through CloudTrail. For AWS services on EC2, I normally prefer an attached instance role instead of manually calling AssumeRole unless role switching is required.

Load Balance

Q20. Explain an architecture with an ALB behind multiple EC2 servers.

Answer: An Application Load Balancer is internet-facing or internal and deployed across multiple Availability Zones. Its listener accepts HTTP/HTTPS traffic and forwards requests according to listener rules to a target group containing EC2 instances on the application port. Health checks determine which targets can receive traffic. EC2 instances usually sit in private subnets, with security groups allowing the application port only from the ALB security group. TLS can terminate at the ALB, and Auto Scaling maintains capacity. I monitor target health, latency, request count and 4xx/5xx errors.

Q21. One EC2 instance is unhealthy. What do you do?

Answer: I check the target group's health-check reason, configured path/port/protocol, timeout and expected response code. On the instance, I verify the application process, listening port, OS and application logs, resource usage, security-group rules, NACLs and connectivity to dependencies. The ALB stops routing new requests to unhealthy targets, but I confirm healthy capacity is sufficient. I replace or recover the instance through the Auto Scaling group if appropriate, then fix the root cause and validate health checks before considering it resolved.

RDS

Q22. Production RDS suddenly becomes slow. How do you troubleshoot?

Answer: I establish the impact and start time, then review CloudWatch metrics such as CPU, freeable memory, connections, read/write latency, IOPS and storage queue depth. I inspect Performance Insights/Database Insights if enabled, slow query logs, execution plans, locks, long-running queries, connection pool behaviour and recent application/schema changes. I check storage capacity and instance limits, network latency and failover events. I mitigate safely—for example, optimize a query or reduce connection pressure—before scaling resources. I validate backups, Multi-AZ/failover status and change controls; I avoid making a production change without measuring its impact.

Q23. Where do you store database passwords and application credentials?

Answer: I use AWS Secrets Manager for secrets that need managed rotation and retrieval, or Systems Manager Parameter Store for suitable configuration and secrets use cases. Access is granted through IAM roles with least privilege, and secrets are encrypted with KMS. The application retrieves secrets at runtime or through a controlled deployment process; credentials are not committed to source control, baked into images, or exposed in logs/user data. I rotate credentials, restrict network access to the database, and audit access.

Cloud Watch

Q24. A production application is erroring. How do you set up monitoring?

Answer: I collect application and system logs in CloudWatch Logs using the CloudWatch agent or application integration, create log groups with retention and appropriate access controls, and define metric filters or embedded metrics for important error patterns. I create dashboards and alarms for error rate, latency, availability, CPU/memory and dependency failures, and route notifications through SNS or an incident-management integration. I define actionable thresholds, test alarms, and include runbooks and deployment annotations so alerts help identify root cause rather than just create noise.

Q25. How do you send automatic alerts when CPU exceeds 80%?

Answer: I open CloudWatch, select the EC2 CPUUtilization metric for the correct instance or Auto Scaling group, and create an alarm with a threshold greater than 80%. I choose an evaluation period and number of datapoints that match the operational need, select the ALARM state action, and send it to an SNS topic with confirmed email or other subscribers. I test the notification path and tune the threshold to avoid transient false alarms. For fleet-level monitoring, I use dimensions or aggregate metrics appropriate to the workload.

Q26. CloudWatch Logs vs Metrics?

Answer: CloudWatch Logs stores log events such as application messages, errors and system logs, which can be searched and analyzed with Logs Insights. CloudWatch Metrics stores timestamped numerical measurements such as CPU utilization, request count, latency and custom application values; metrics are used for dashboards and alarms. Logs can be converted into metrics using metric filters or embedded metric format. In practice, I use metrics to detect that something is wrong and logs to investigate why.

SNS

Q27. Explain the architecture of CloudWatch alarms with SNS.

Answer: An AWS service or application publishes metrics to CloudWatch. A CloudWatch alarm evaluates a metric against a configured threshold over specified periods and changes state to OK, ALARM or INSUFFICIENT_DATA. For the ALARM state, an alarm action publishes a notification to an SNS topic. SNS then delivers the message to confirmed subscribers such as email, SMS where supported, HTTPS endpoints, or an integrated incident-management service. I verify topic permissions, subscription confirmation, alarm history and delivery status, and avoid including secrets in notifications.

Q28. How do you create an SNS topic?

Answer: In the SNS console, I create a Standard topic for ordinary fan-out notifications or a FIFO topic when its ordering/deduplication characteristics are required and supported by the workflow. I configure access policy and encryption if required, add a subscription such as email or HTTPS, and confirm the subscription. Then I connect the topic to a CloudWatch alarm or publish a test message. I validate delivery, restrict who can publish/subscribe, and monitor delivery failures where applicable.

Cloud trail

Q29. In production, who deleted an AWS service/resource and how do you identify them?

Answer: I use AWS CloudTrail event history to search management events around the deletion time, filtering by event name, resource name, user identity and time. The event record can show the API action, principal or assumed role, source IP, event time, user agent and request details, depending on the event. For longer retention and centralized investigation, I use a trail or CloudTrail Lake configured for the required events and deliver logs to a protected S3 bucket, ideally with restricted deletion and appropriate integrity controls. I correlate with AWS Config, CloudWatch and deployment/audit records. CloudTrail identifies the API principal; for assumed roles, I trace the session to the originating identity. I also preserve evidence and follow the incident process.

Q30. CloudTrail vs CloudWatch?

Answer: CloudTrail records AWS account activity and API events—who performed an action, what action occurred, when, and relevant request context—so it is primarily for audit, governance and investigation. CloudWatch collects metrics, logs and events and supports dashboards, alarms and operational monitoring. For example, CloudTrail can help identify who deleted an EC2 instance, while CloudWatch can alert that CPU is high or show application error logs. They complement each other.